From Circuits to Systems: A Practical Roadmap to Embedded Engineering

This book is a layered, systems-first guide to electronics, embedded systems, single board computers, operating systems, PCB design, ESP32 development, and reverse engineering. It is written to mirror how real hardware and software systems actually work in the physical world.

Chapter 1: Electricity & Electronics Foundations

Voltage, current, resistance, and power form the physical substrate of all computing systems.

Learn Ohm’s Law, Kirchhoff’s laws, and how electrons behave in conductors.

Understand resistors, capacitors, inductors, diodes, and transistors.

Tip: If you cannot predict voltage at a node, stop and analyze before writing code.

Chapter 2: Digital Logic & Microcontroller Fundamentals

Digital systems abstract analog reality into binary logic.

Learn logic gates, truth tables, clocks, registers, and GPIO.

Understand interrupts, timers, and hardware state machines.

Tip: A microcontroller is a deterministic machine. Treat it as such.

Chapter 3: Embedded C & Memory

Embedded C is about controlling memory and hardware directly.

Pointers, memory-mapped registers, and volatile keywords are essential.

Learn stack vs heap, linker scripts, and startup code.

Tip: Undefined behavior is not theoretical. It breaks devices.

Chapter 4: ESP32 & Modern Embedded Systems

The ESP32 blends MCU design with networking stacks.

Understand FreeRTOS tasks, Wi-Fi/Bluetooth subsystems, and flash partitions.

Learn bootloaders, OTA updates, and power modes.

Tip: Concurrency bugs are the hardest bugs.

Chapter 5: PCB Design & Hardware Integration

Schematics describe intent. Layout determines reality.

Learn decoupling, ground planes, trace impedance, and EMI.

Understand manufacturing constraints and assembly tolerances.

Tip: Every PCB revision is an education tax.

Chapter 6: Single Board Computers & Linux

SBCs differ from MCUs by running full operating systems.

Learn the Linux boot process, device trees, and kernel modules.

Understand userspace vs kernel space.

Tip: Linux hides complexity but never removes it.

Chapter 7: Operating System Internals

Operating systems manage time, memory, and isolation.

Learn scheduling, virtual memory, filesystems, and syscalls.

Understand why performance problems are systemic.

Tip: The OS is a referee, not a magician.

Chapter 8: Debugging & Instrumentation

Debugging is observing reality, not guessing.

Learn oscilloscopes, logic analyzers, JTAG, and GDB.

Master reading datasheets and probing signals.

Tip: Measure first. Assume later.

Chapter 9: Reverse Engineering Embedded Systems

Reverse engineering reveals how systems actually behave.

Learn firmware extraction, disassembly, and decompilation.

Understand boot ROMs, flash layouts, and undocumented interfaces.

Tip: Reverse engineering is applied curiosity.